

A&I

android and ios develop.

SASRec

2026 A&I 4th

구승민

목차

1. 코드 실습

- Attention·Transformer를 라이브러리로 직접 만들기

2. 코드 설명

- 실습에서 활용된 코드의 동작 이해하기

3. SASRec

- 구현 방식에 따른 모델의 결과 확인하기

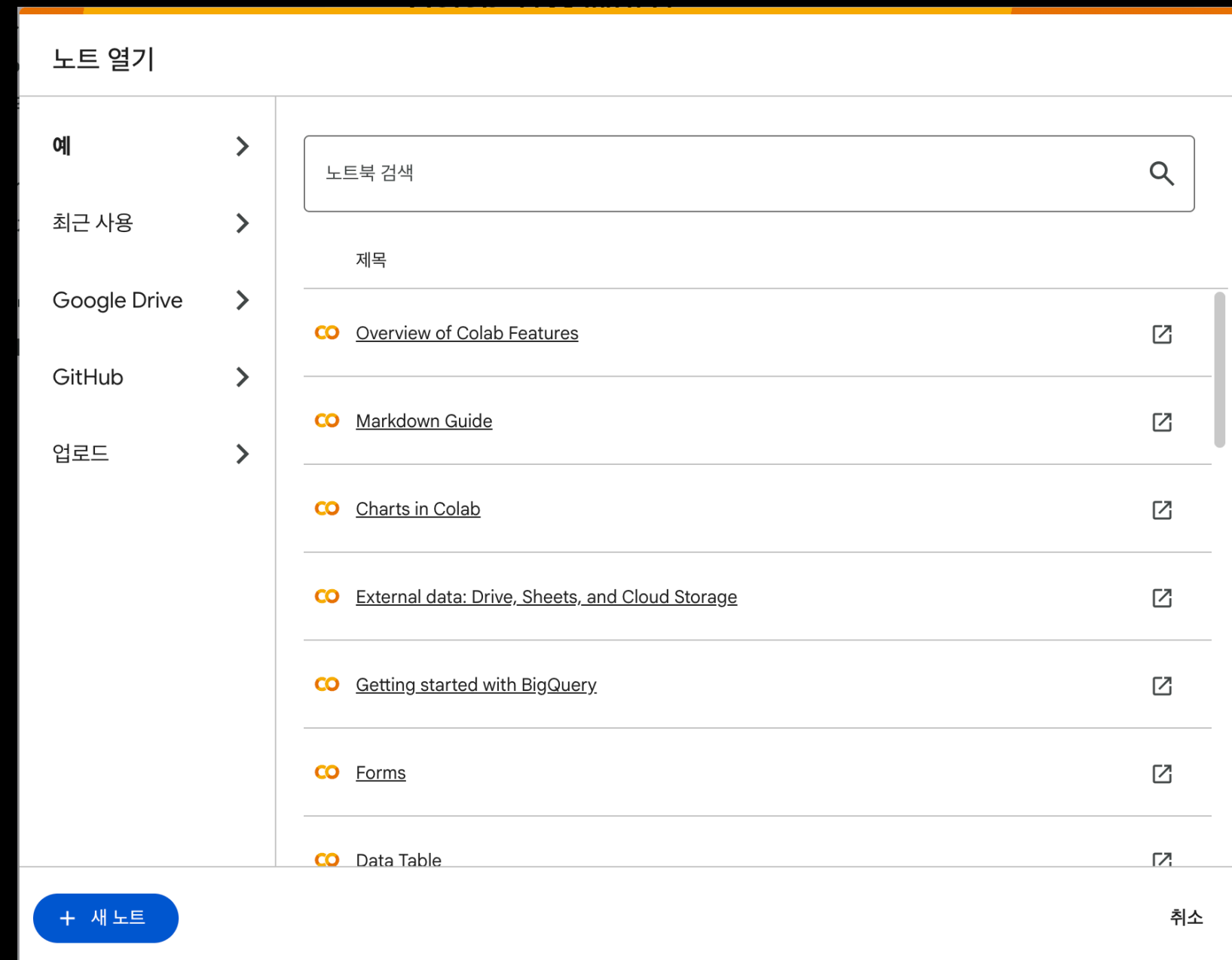
1. 코드 실습

- 그냥 따라하시면 됩니다

실습 환경 구성하기

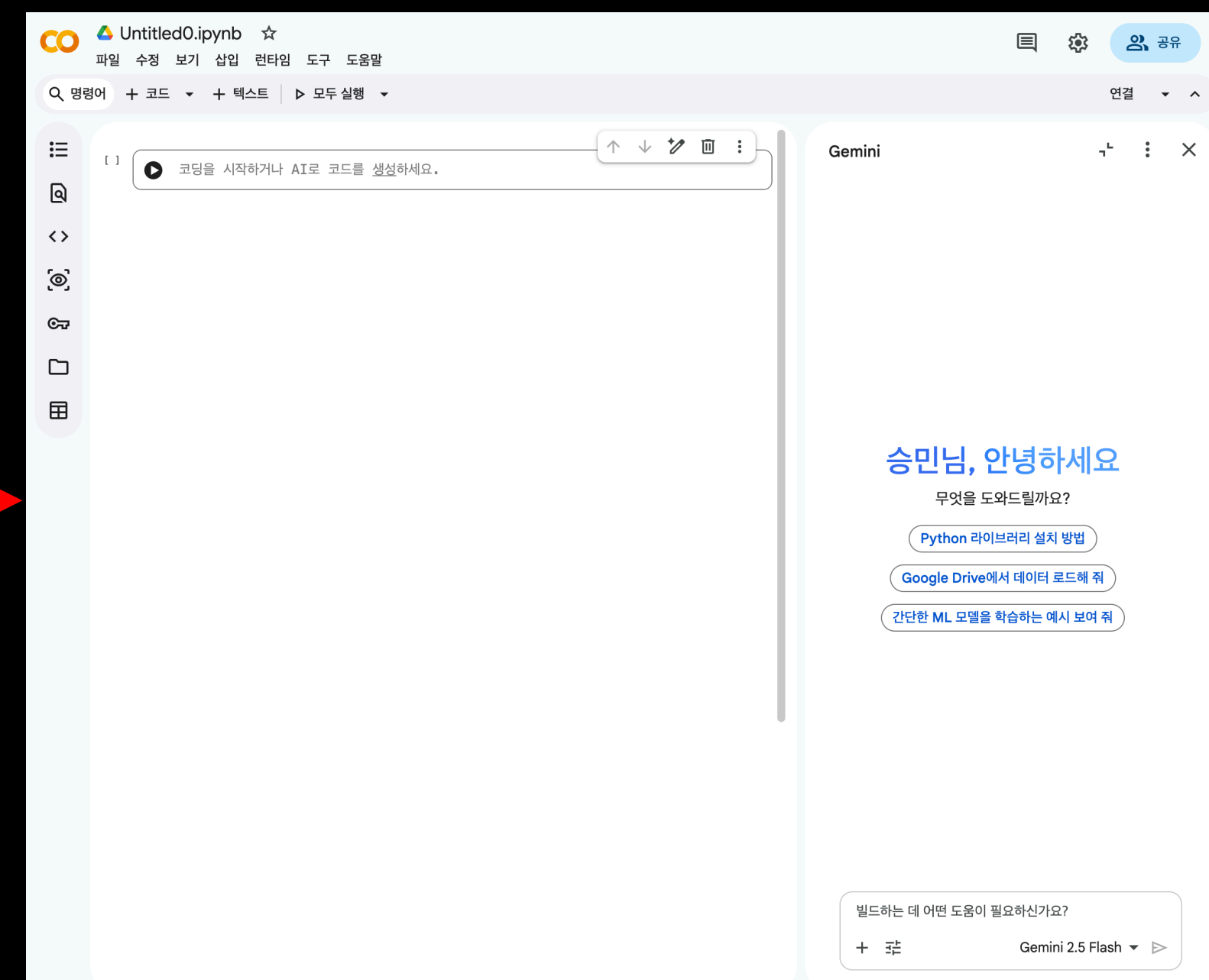
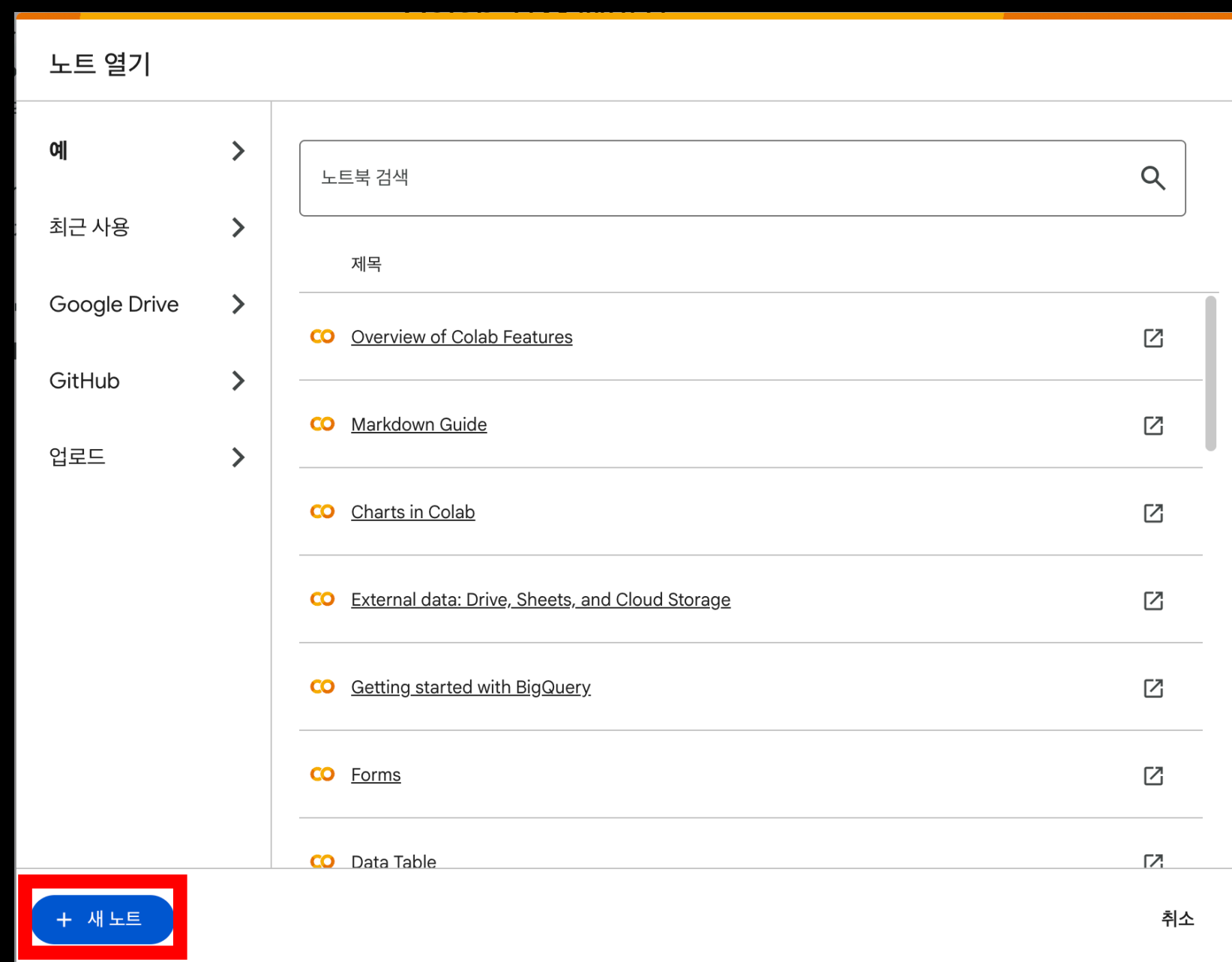
아래 링크로 이동

- <https://colab.research.google.com>



실습 환경 구성하기

하단 새 노트 클릭하여 코드 환경 구성



코랩 단축키

[실행 관련 단축키]

1. Command + Enter

- 결과 값만 보고자 할 때
- 해당 셀을 실행하고 커서를 해당 셀에 두는 경우

2. Shift + Enter

- 여러가지 값을 빠르게 출력할 때
- 해당 셀을 실행하고 커서를 다음 셀로 넘기는 경우

3. Alt + Enter

- 다음 작업 공간이 없을 때
- 해당 셀을 실행하고 셀을 삽입한 후 커서를 삽입한 셀로 넘기는 경우

코랩 단축키

[셀 삽입/삭제 관련 단축키]

1. Command + M D

- 셀 지우기

2. Command + M Y

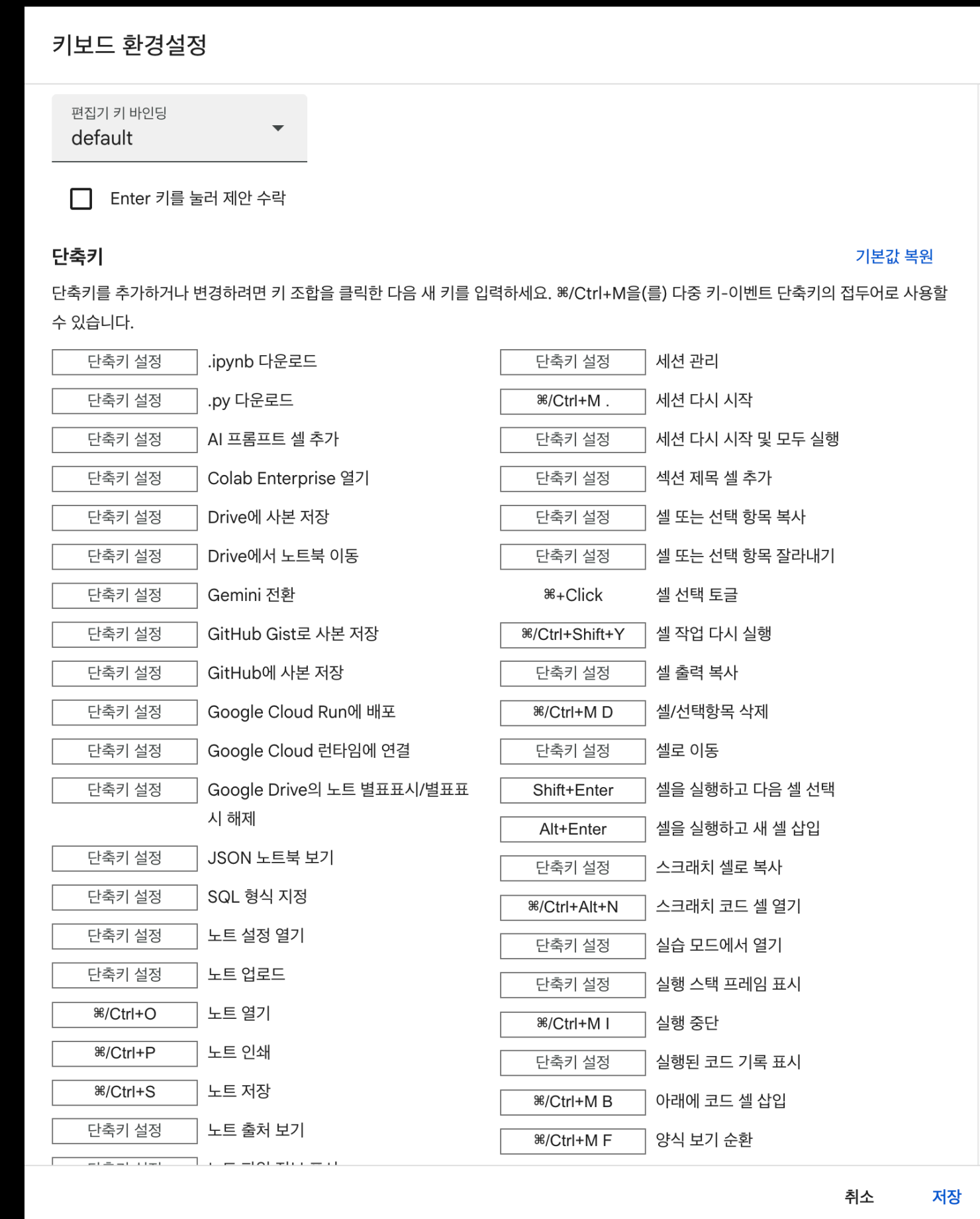
- 코드 셀로 변경

3. Command + M M

- 마크다운 셀로 변경

4. Command + M H

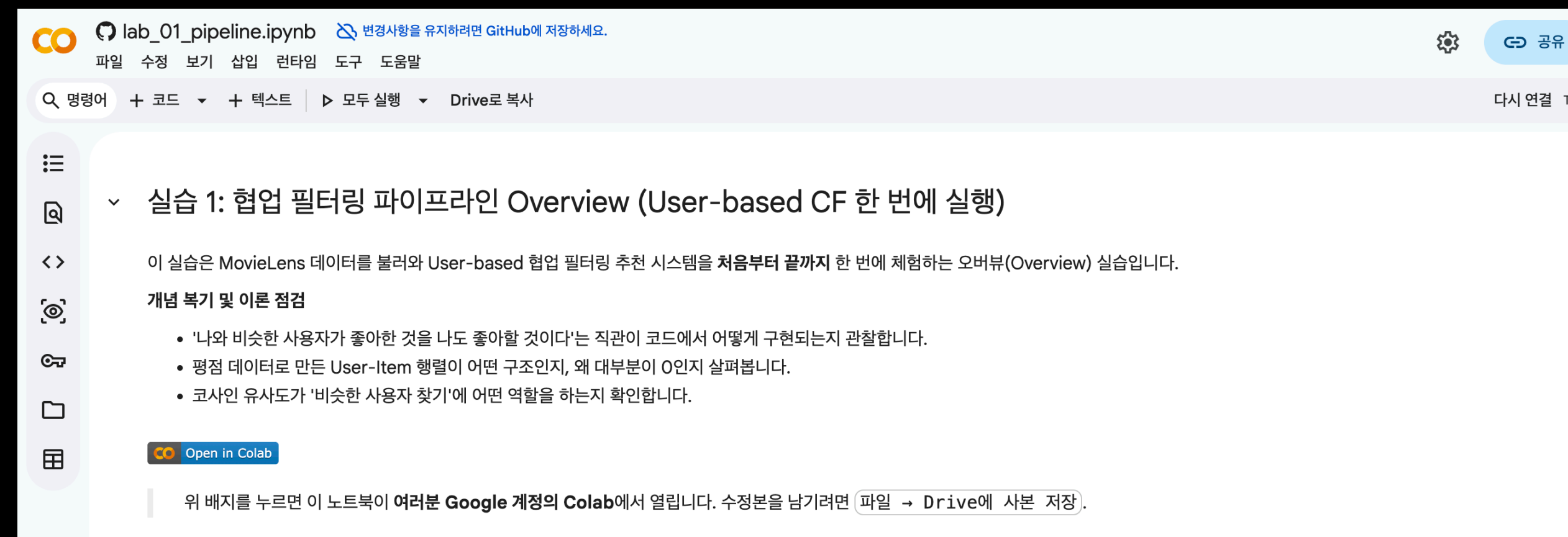
- 단축키 모음 열기



코드 실습

코드 내용을 따라하며 실습

- 중간에 놓치는 경우 아래 링크에서 내용을 확인
- https://colab.research.google.com/github/Team-Anl/A-AND-I-4TH-AI-CODE-LAB/blob/main/4%EC%A3%BC%EC%B0%A8/lab_01_pipeline.ipynb



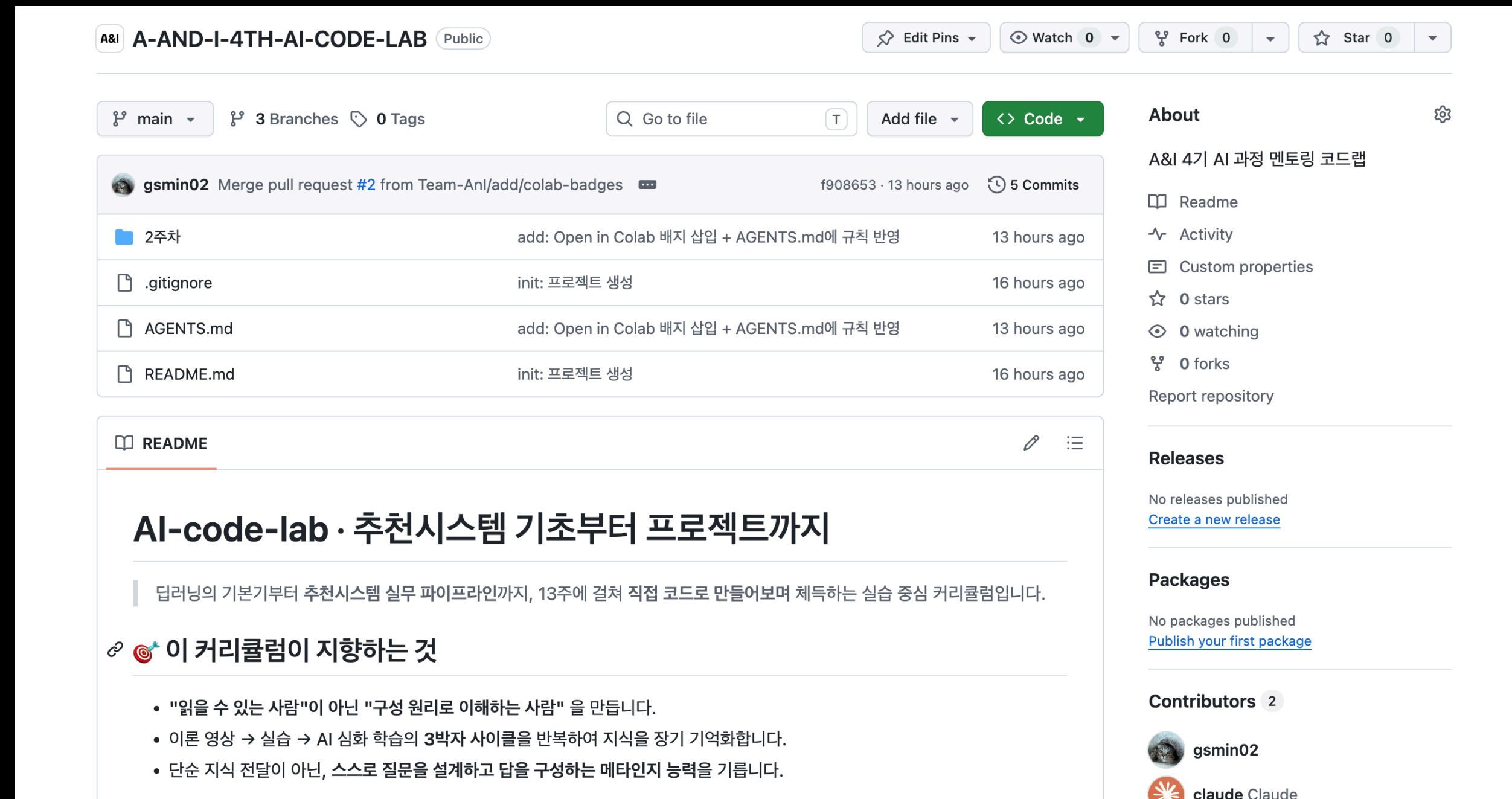
2. 코드 설명

- 작성한 코드의 세부 내용을 살펴봅니다

강의자료 확인 방법

강의자료는 아래 링크를 통해 확인 가능

- <https://github.com/Team-Anl/A-AND-I-4TH-AI-CODE-LAB>



코드 실습 수행

각 실습은 제공된 ipynb 파일 상단 “**Open in Colab**”으로 실습 가능

The screenshot shows a GitHub repository page for the file 'lab_01_tensor_basics.ipynb'. The file is 327 lines (327 loc) and 10.1 KB. The 'Preview' tab is selected, showing the title '실습 1: Tensor와 딥러닝 데이터 핸들링 (The Foundation)'. Below the title is a description: '목표: 딥러닝의 기본 데이터 단위인 Tensor를 자유자재로 다루고, 실제 환경에 맞춰 데이터를 효율적으로 전송하는 방법을 이해한다.' followed by a section '개념 복기 및 이론 점검' with three points: 1. Tensor와 NumPy 배열의 근본적인 차이점 (GPU 지원, 자동 기울기 추적 가능 여부), 2. 딥러닝에서 데이터의 형태(Shape)가 왜 중요한가 (입력 차원의 일치), 3. dtype 과 device 개념. At the bottom, there is a red box highlighting the 'Open in Colab' button. A red arrow points from this button to the Colab interface on the right.

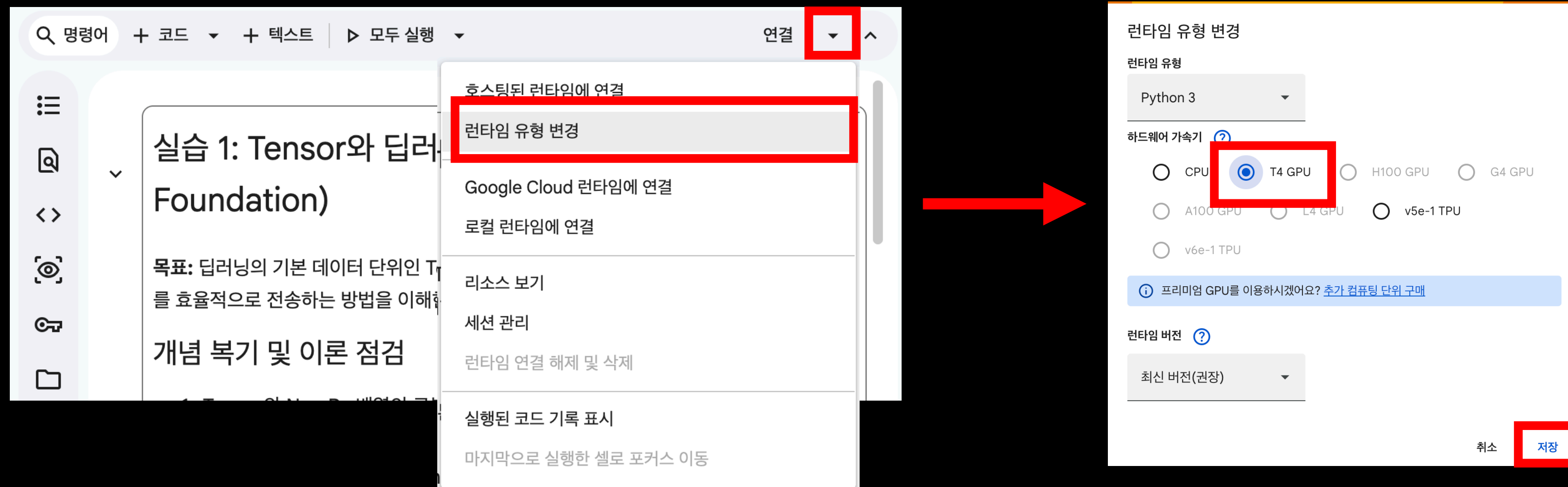
The screenshot shows the Google Colab interface for the file 'lab_01_tensor_basics.ipynb'. The title '실습 1: Tensor와 딥러닝 데이터 핸들링 (The Foundation)' is at the top. Below it is the same description as in the GitHub preview. The '개념 복기 및 이론 점검' section is also present. At the bottom, there is a code cell with the following code:

```
import torch
import numpy as np

print(f"PyTorch 버전: {torch.__version__}")
print(f"CUDA 사용 가능 여부: {torch.cuda.is_available()}")
print(f"MPS(Apple Silicon) 사용 가능 여부: {torch.backends.mps.is_available()}")
```

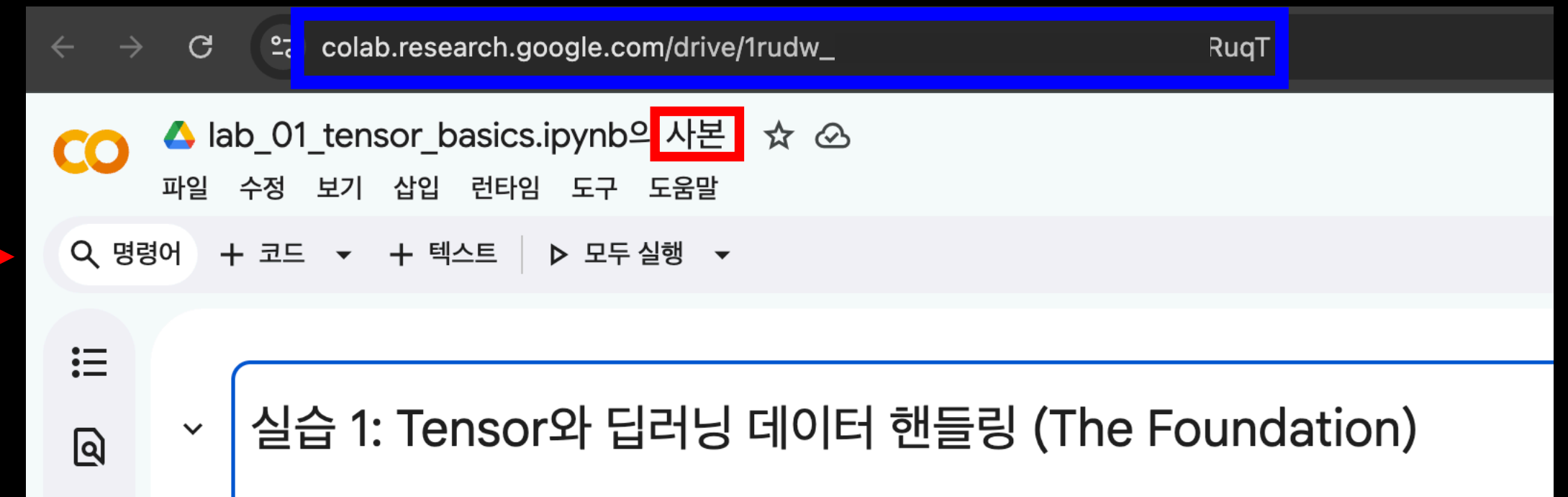
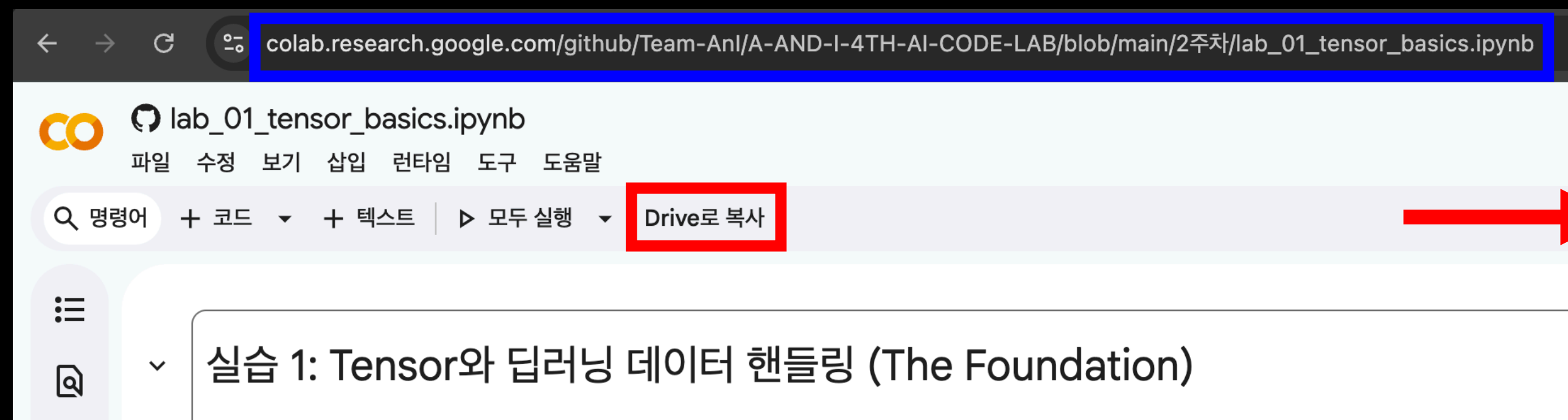

코드 실습 주의사항

실행 전 “런타임 유형 변경”을 통해 “T4 GPU”로 변경해야 함



내용 편집

내용을 변경하고 싶은 경우 “**Drive로 복사**”를 통해 편집 가능



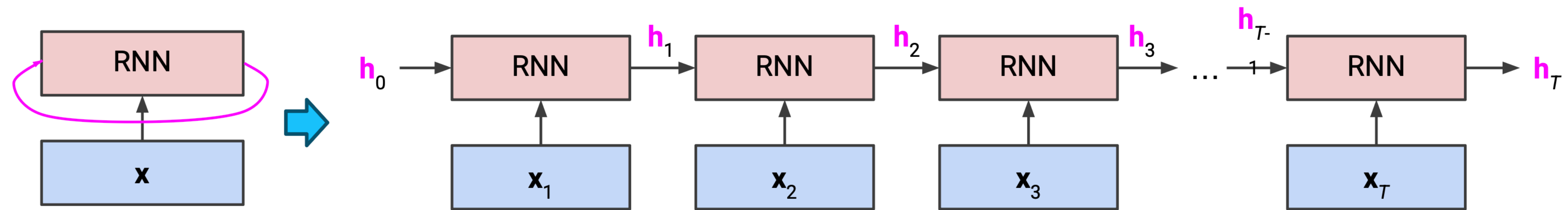
1. 환경 준비: 라이브러리 импорт

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

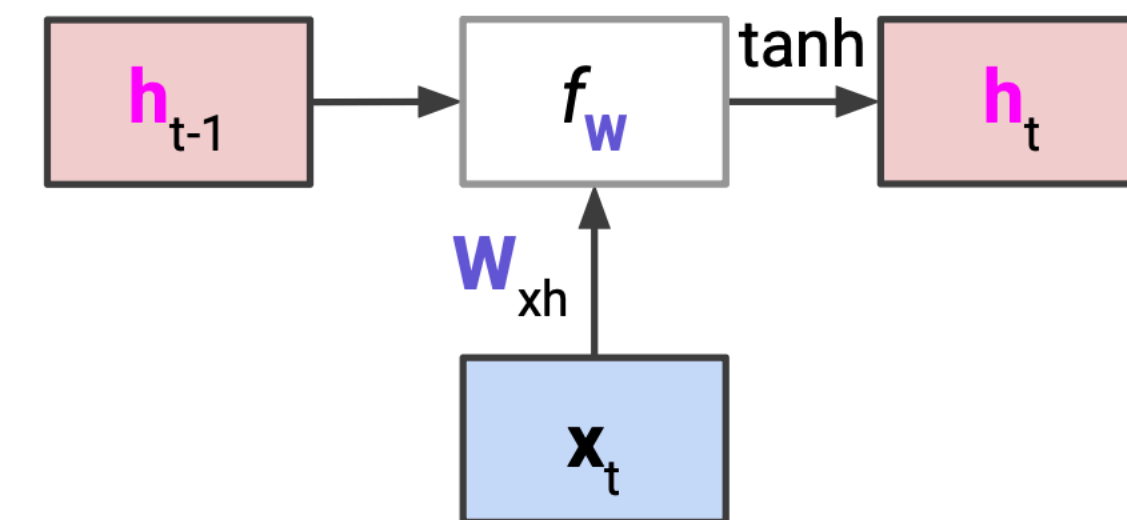
torch.manual_seed(42)
device = "cuda" if torch.cuda.is_available() else (
    "mps" if torch.backends.mps.is_available() else "cpu")
print("device:", device)
```

코드 해설

- `torch.nn` : 어텐션·트랜스포머·임베딩 등 신경망 구성요소가 모여 있는 모듈입니다. 이번 주치의 주인공인 `MultiheadAttention`, `TransformerEncoderLayer` 가 여기 들어 있습니다.
- `torch.manual_seed(42)` : 임베딩 등 가중치가 무작위로 초기화되므로, 시드를 고정해 매 실행 결과를 재현 가능하게 만듭니다.
- `device` 자동 선택: GPU(`cuda`)·애플 실리콘(`mps`)이 있으면 사용하고, 없으면 CPU로 떨어집니다. 이번 toy 예제는 작아서 CPU로도 즉시 돌아갑니다.



$$h_t = f_w(h_{t-1}, x_t)$$



2. 입력 시퀀스 준비: 아이템 임베딩 + 위치 임베딩

한 사용자가 순서대로 본 아이템 시퀀스를 임베딩으로 바꾸고, 순서를 알려주는 위치 임베딩을 더합니다.

```
# 한 사용자가 순서대로 소비한 아이템 시퀀스 (toy 예시)
num_items = 20          # 전체 아이템 종류 수
d_model = 16            # 임베딩 차원
seq = torch.tensor([[3, 7, 1, 9, 4]]) # (batch=1, seq_len=5)
seq_len = seq.size(1)

item_emb = nn.Embedding(num_items, d_model)
pos_emb = nn.Embedding(seq_len, d_model)

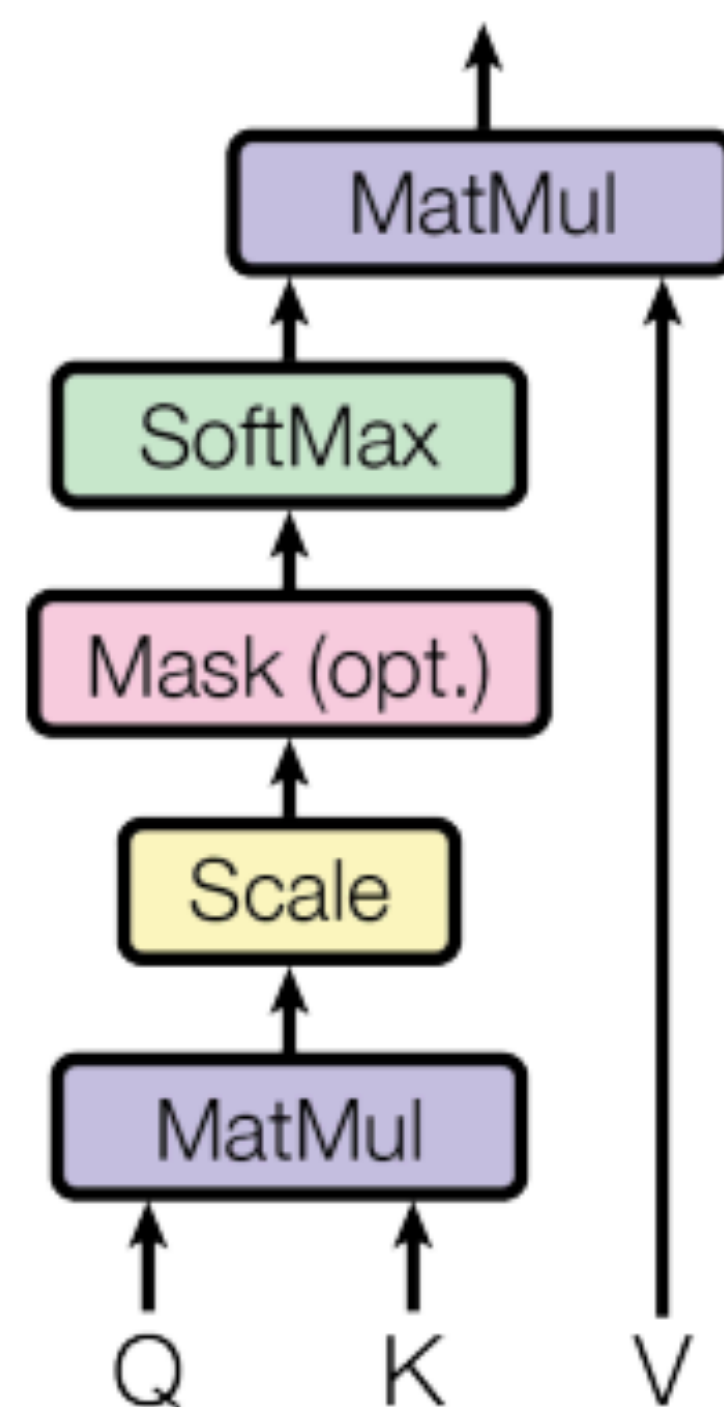
positions = torch.arange(seq_len).unsqueeze(0) # (1, seq_len)
x = item_emb(seq) + pos_emb(positions)         # (1, seq_len, d_model)

print("입력 임베딩 shape:", x.shape)
```

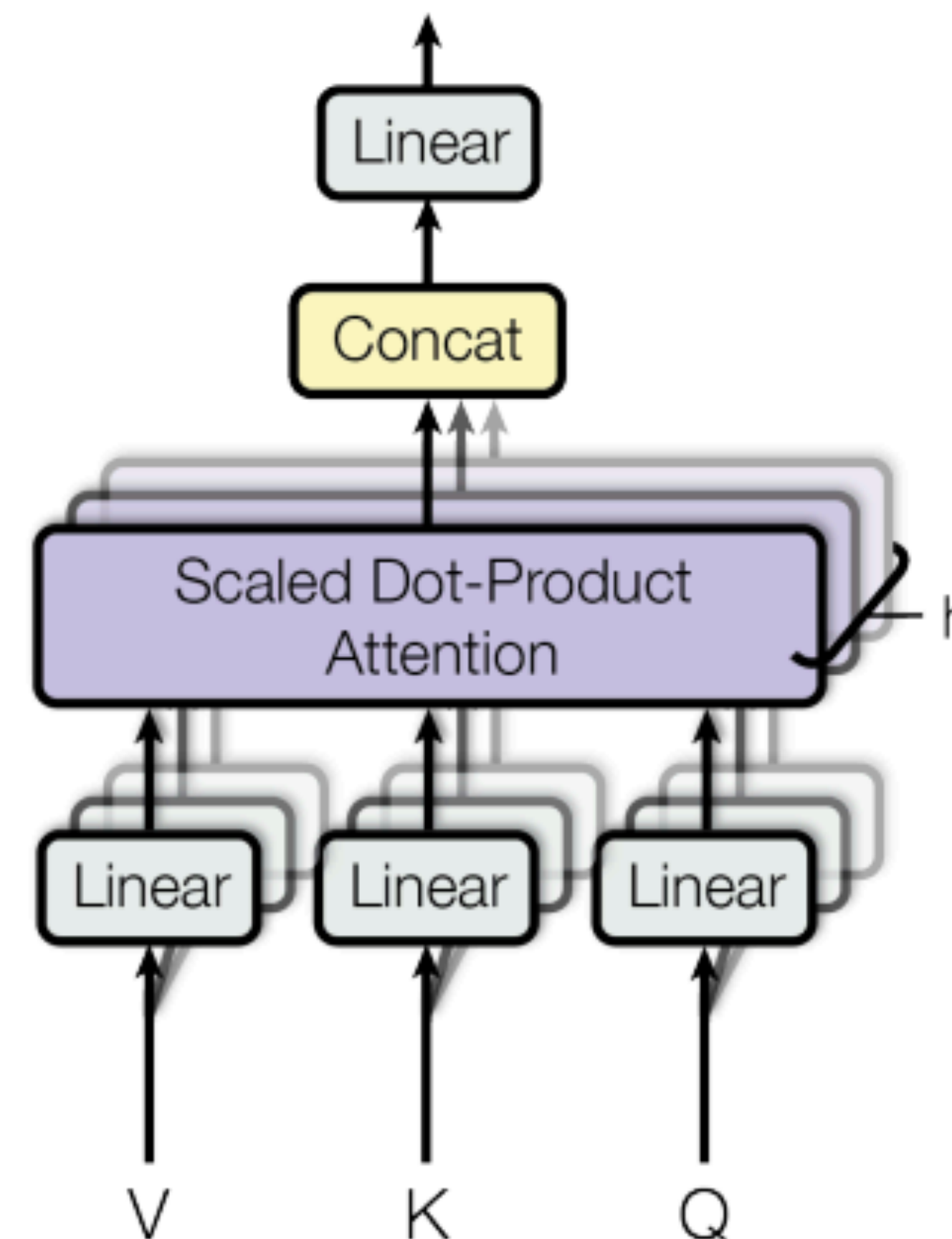
코드 해설

- **nn.Embedding(num_items, d_model)** : 아이템 ID(정수)를 **d_model** 차원의 벡터로 변환하는 조회 테이블입니다. 8주차에서 배운 **임베딩**이 그대로 쓰입니다 — 아이템 ID는 그 자체로 의미가 없지만, 학습을 통해 비슷한 아이템이 가까운 벡터를 갖게 됩니다.
- **pos_emb (위치 임베딩)**: 셀프 어텐션은 모든 위치를 동시에 보기 때문에 **순서를 모릅니다**. 그래서 '몇 번째 위치인가'를 나타내는 벡터를 따로 더해 순서 정보를 주입합니다. (트랜스포머 원논문은 sin/cos 함수를, SASRec은 학습형 위치 임베딩을 씁니다. 여기서는 후자입니다.)
- **item_emb(seq) + pos_emb(positions)** : '무엇을 봤는가(아이템)'와 '언제 봤는가(위치)'를 더해 하나의 입력 표현으로 만듭니다. 결과 shape **(1, 5, 16)** = (배치, 시퀀스 길이, 차원).

Scaled Dot-Product Attention



Multi-Head Attention



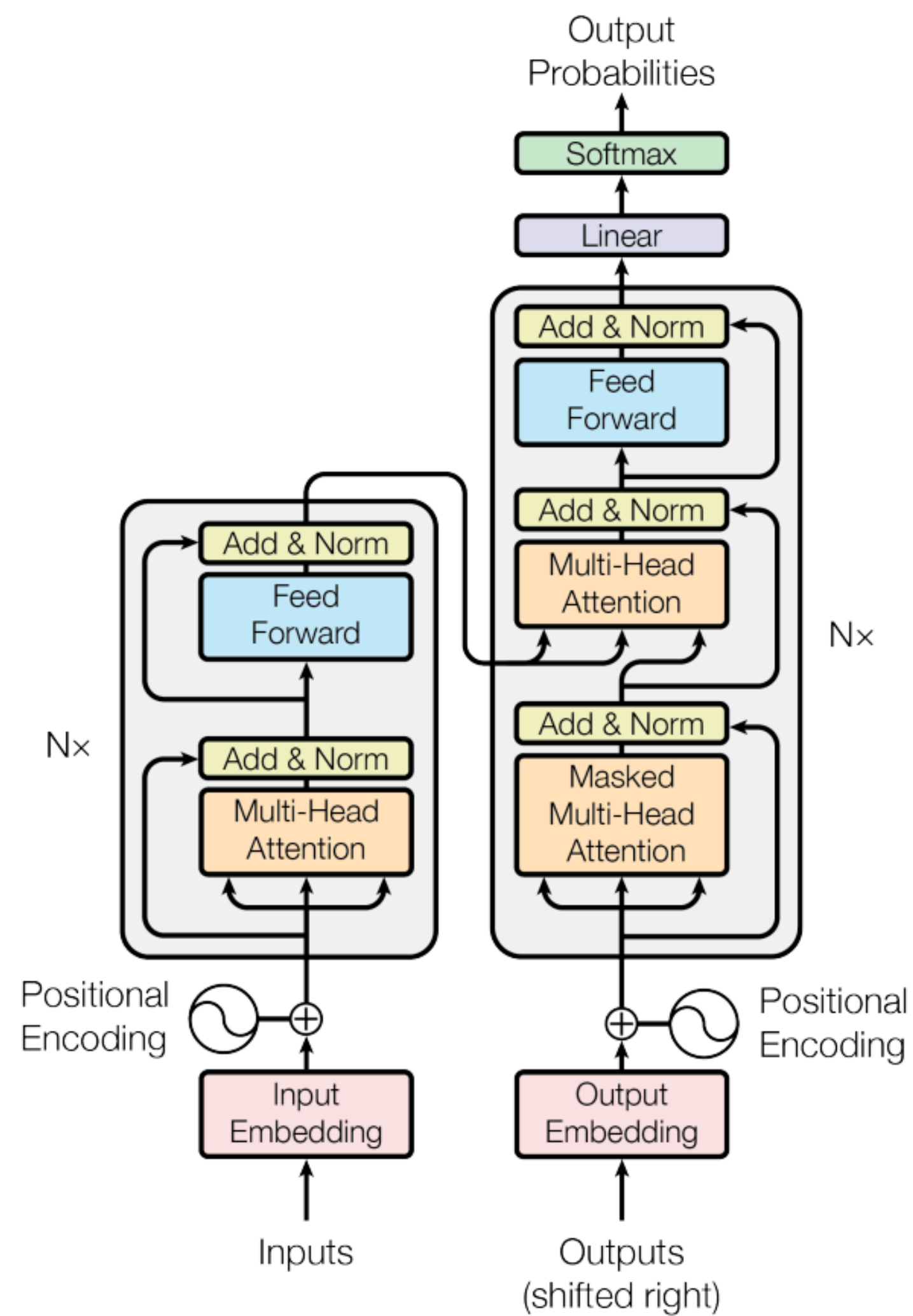
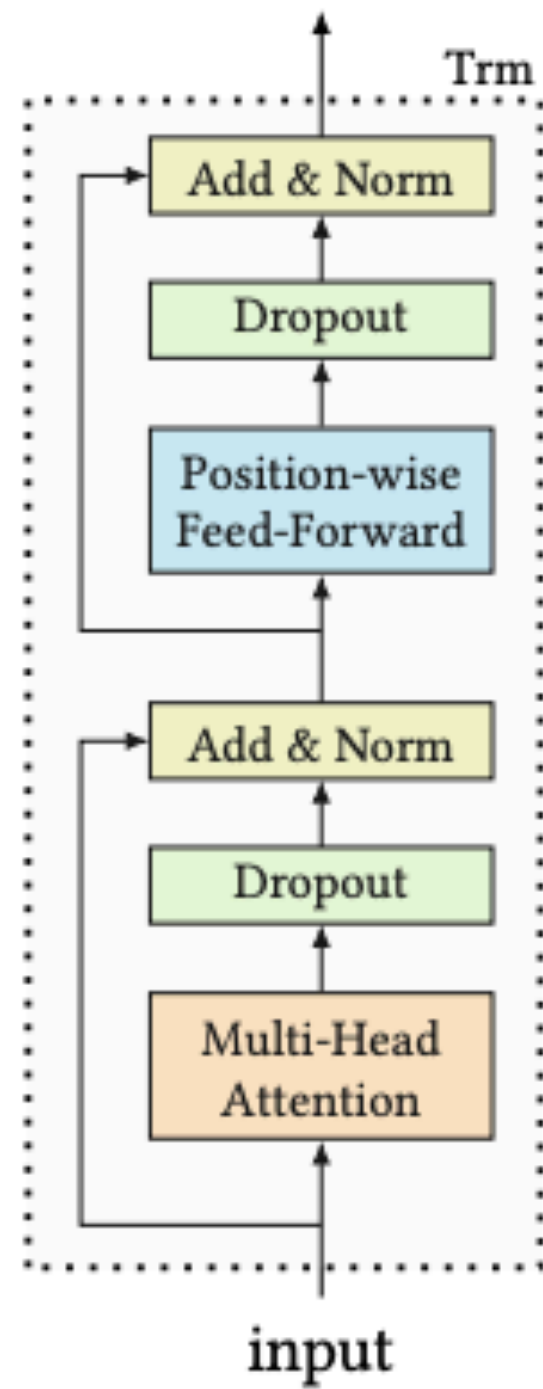
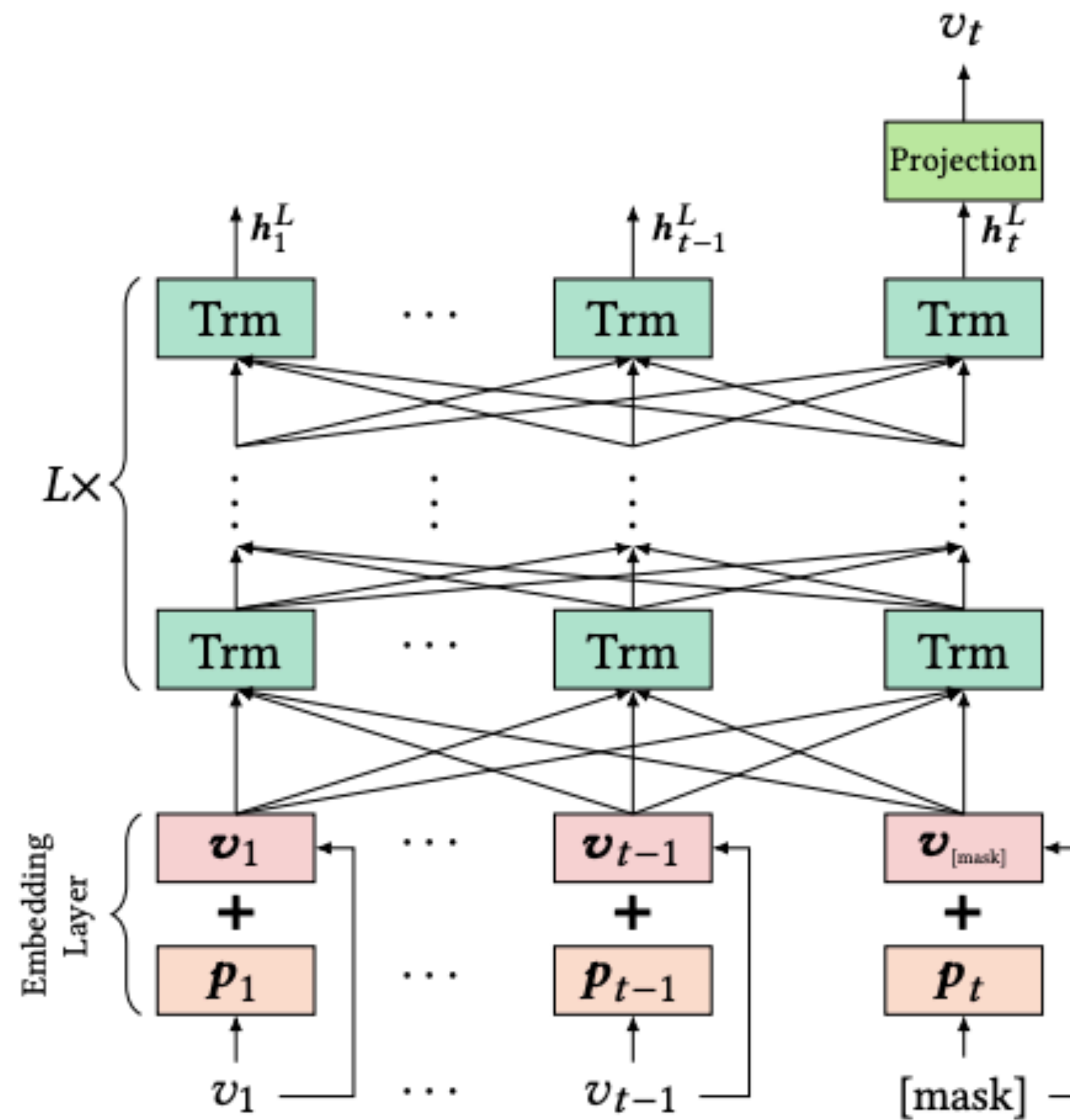


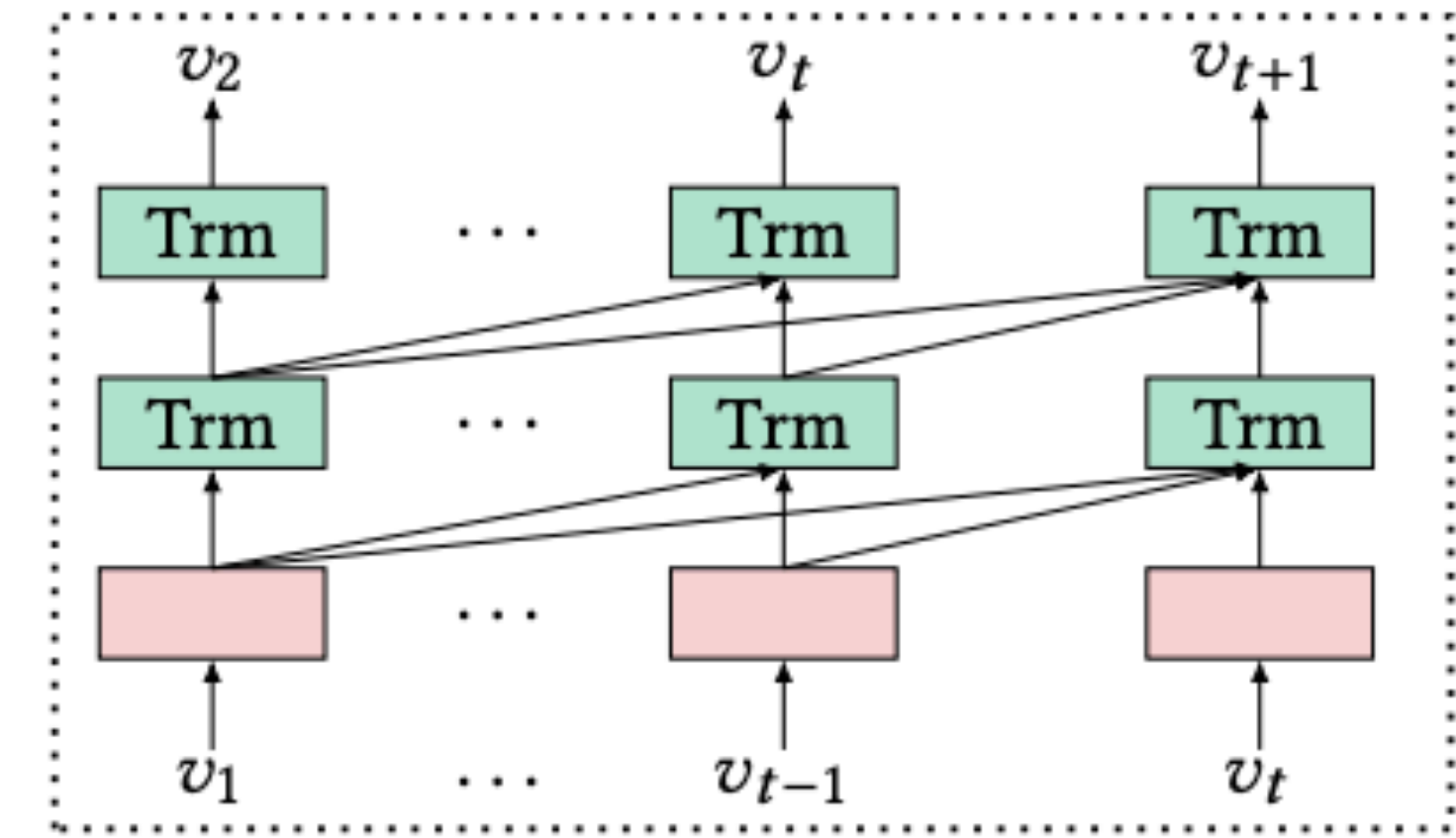
Figure 1: The Transformer - model architecture.



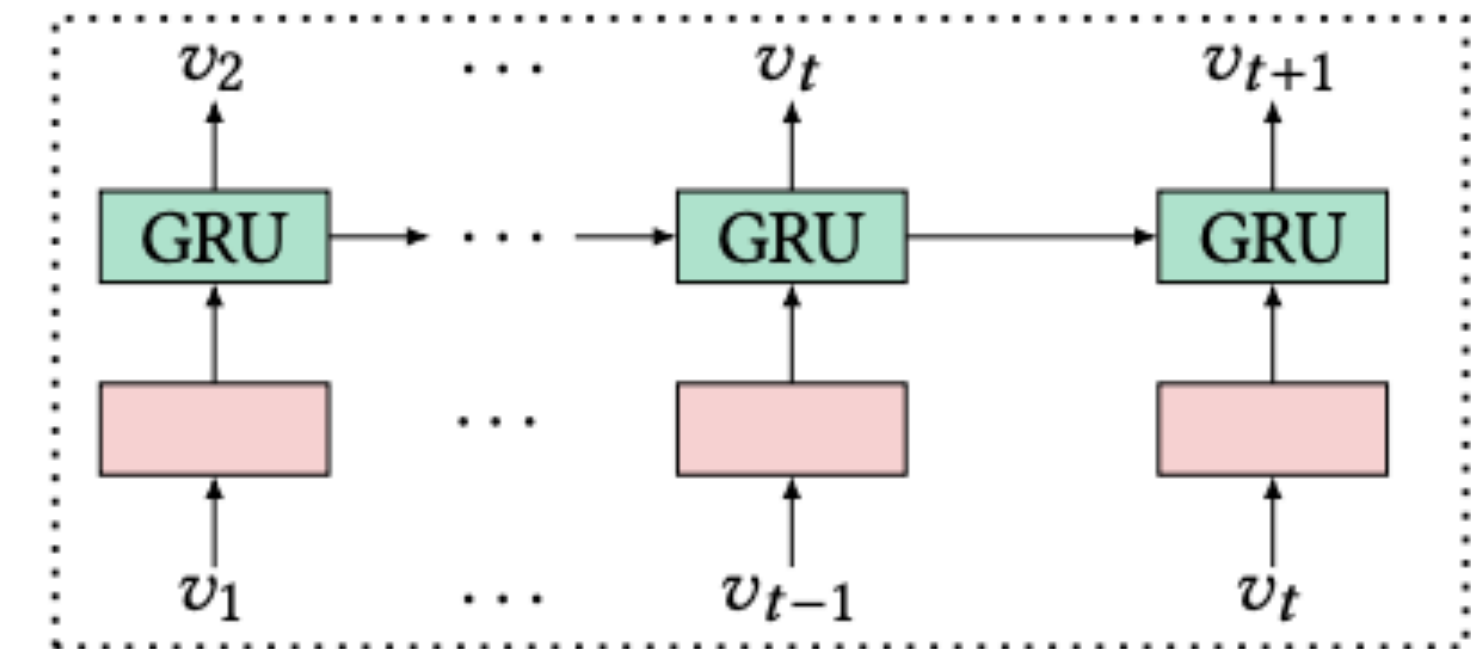
(a) Transformer Layer.



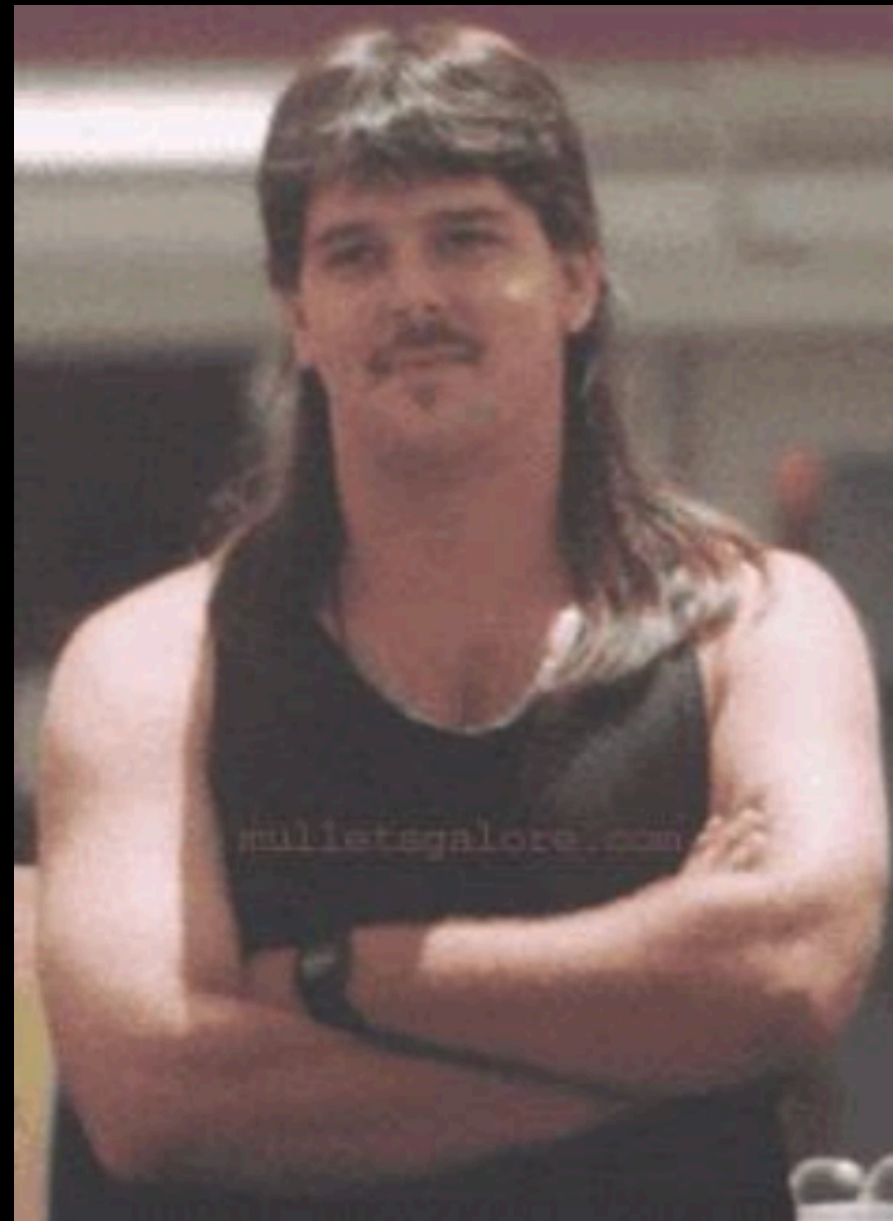
(b) BERT4Rec model architecture.



(c) SASRec model architecture.



(d) RNN based sequential recommendation methods.



?

- `nn.Embedding(num_items, d_model)` : 아이템 ID(정수)를 `d_model` 차원의 벡터로 변환하는 조회 테이블입니다. 8주차에서 배운 임베딩이 그대로 쓰입니다 — 아이템 ID는 그 자체로 의미가 없지만, 학습을 통해 비슷한 아이템이 가까운 벡터를 갖게 됩니다.

- **pos_emb (위치 임베딩):** 셀프 어텐션은 모든 위치를 동시에 보기 때문에 **순서를 모릅니다**. 그래서 '몇 번째 위치인가'를 나타내는 벡터를 따로 더해 순서 정보를 주입합니다. (트랜스포머 원논문은 sin/cos 함수를, SASRec은 학습형 위치 임베딩을 씁니다. 여기서는 후자입니다.)

- `item_emb(seq) + pos_emb(positions)` : '무엇을 봤는가(아이템)'와 '언제 봤는가(위치)'를 더해 하나의 입력 표현으로 만듭니다. 결과 shape `(1, 5, 16)` = (배치, 시퀀스 길이, 차원).

3. 셀프 어텐션 (nn.MultiheadAttention)

각 위치가 모든 위치를 바라보며 관계(어텐션 가중치)를 계산합니다.

```
mha = nn.MultiheadAttention(embed_dim=d_model, num_heads=2, batch_first=True)

attn_out, attn_w = mha(x, x, x, need_weights=True)

print("어텐션 출력 shape:", attn_out.shape)    # (1, seq_len, d_model)
print("어텐션 가중치 shape:", attn_w.shape)     # (1, seq_len, seq_len)
print("\n어텐션 가중치(각 행의 합 = 1):\n", attn_w[0].detach().round(decimals=2))
```

코드 해설

- **nn.MultiheadAttention(embed_dim, num_heads, batch_first=True)** : 셀프 어텐션을 한 줄로 제공하는 모듈입니다. 내부에서 입력을 Query·Key·Value로 선형 변환하고, $\text{softmax}(QK^T / \sqrt{d_k})V$ 를 계산합니다. **num_heads=2** 는 **멀티 헤드** — 서로 다른 관점의 관계를 2개의 헤드로 동시에 학습합니다. **batch_first=True** 면 입력 shape이 (배치, 길이, 차원) 이 됩니다.
- **mha(x, x, x)** : Query·Key·Value 자리에 **같은 입력 x** 를 세 번 넣는 것이 바로 '셀프(self)' 어텐션입니다. 시퀀스가 자기 자신을 바라보며 내부 관계를 학습합니다.
- **attn_w (어텐션 가중치, shape (1, 5, 5))** : **attn_w[0][i][j]** 는 i번째 위치가 j번째 위치에 둔 가중치입니다. 각 **행의 합이 1**(softmax 결과)이며, 한 행은 '이 위치가 다른 위치들을 어떤 비율로 참고했는가'를 나타냅니다.

4. Causal Masking: 미래 가리기

다음 아이템을 예측하려면 아직 일어나지 않은 미래를 참조하면 안 됩니다. 상삼각을 $-\infty$ 로 막습니다.

```
causal_mask = torch.triu(
    torch.full((seq_len, seq_len), float('-inf')), diagonal=1)

masked_out, masked_w = mha(x, x, x, attn_mask=causal_mask, need_weights=True)

print("causal mask:\n", causal_mask)
print("\n마스킹 후 어텐션 가중치(상삼각=0):\n", masked_w[0].detach().round(decimals=2))
```

코드 해설

- **`torch.triu(..., diagonal=1)`** : 대각선 위쪽(상삼각)만 남기는 함수입니다. 그 자리를 `-inf` 로 채워 마스크를 만듭니다.
- **왜 $-\infty$ 인가**: 어텐션 점수에 이 마스크를 더한 뒤 softmax를 취하면, $e^{-\infty} = 0$ 이 되어 해당 위치의 가중치가 **정확히 0**이 됩니다. 즉 '그 위치는 보지 않는다'가 됩니다.
- **왜 미래를 가리는가**: 추천(혹은 언어 생성)에서 t번째 다음 아이템을 예측할 때, t+1 이후의 아이템은 아직 일어나지 않은 미래입니다. 학습 중 미래를 훑쳐보면 시험 답안을 미리 본 것과 같아 실제 추론 상황과 어긋납니다. 마스킹 후 가중치 행렬의 **상삼각이 모두 0**이 된 것을 확인하세요. 이것이 SASRec이 쓰는 **인과적(causal) 셀프 어텐션**입니다.

5. 어텐션 가중치 시각화

마스킹 전후의 어텐션 가중치를 히트맵으로 비교합니다.

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, w, title in [(axes[0], attn_w, "Self-Attention"),
                    (axes[1], masked_w, "Causal Self-Attention")]:
    im = ax.imshow(w[0].detach(), cmap="YlOrRd", vmin=0, vmax=1)
    ax.set_title(title)
    ax.set_xlabel("Key position (past items)")
    ax.set_ylabel("Query position (current step)")
    ax.set_xticks(range(seq_len)); ax.set_yticks(range(seq_len))
    fig.colorbar(im, ax=ax, fraction=0.046)
plt.tight_layout(); plt.show()
```

코드 해설

- 왼쪽(일반 셀프 어텐션)은 행렬 전체에 값이 분포하지만, 오른쪽(causal)은 대각선 아래쪽만 값을 갖습니다. 각 Query 위치가 자기 자신과 과거만 참고한다는 뜻입니다.
- 가로축(Key)은 '참고 대상이 되는 과거 아이템', 세로축(Query)은 '지금 예측의 기준이 되는 위치'로 읽으면 됩니다. 학습 전이므로 값 자체는 큰 의미가 없지만, **마스킹의 구조적 효과**는 분명히 드러납니다.

6. Transformer 인코더 (nn.TransformerEncoderLayer)

어텐션 + 잔차연결 + LayerNorm + 피드포워드를 묶은 표준 블록을 쌓습니다.

```
encoder_layer = nn.TransformerEncoderLayer(
    d_model=d_model, nhead=2, dim_feedforward=64, batch_first=True)
encoder = nn.TransformerEncoder(encoder_layer, num_layers=2)

out = encoder(x, mask=causal_mask)
print("Transformer 인코더 출력 shape:", out.shape)  # (1, seq_len, d_model)
```

코드 해설

- **nn.TransformerEncoderLayer** : 셀프 어텐션 하나만 있는 게 아니라, ①멀티 헤드 어텐션 → ②잔차연결(residual) + LayerNorm → ③피드포워드 신경망 (dim_feedforward) → ④다시 잔차연결 + LayerNorm 으로 이뤄진 **한 블록**입니다. 잔차연결과 정규화는 층을 깊게 쌓아도 학습이 안정되도록 돕습니다.
- **nn.TransformerEncoder(layer, num_layers=2)** : 위 블록을 2번 쌓습니다. SASRec도 보통 이런 블록을 2개 적층합니다.
- **mask=causal_mask** : 인코더 전체에 동일한 인과 마스크를 적용해, 모든 층에서 미래를 보지 못하게 합니다.

7. 다음 아이템 예측 헤드

각 시점의 출력으로 다음에 올 아이템 점수를 만듭니다.

```
head = nn.Linear(d_model, num_items)
logits = head(out)                                # (1, seq_len, num_items)
next_item = logits[:, -1].argmax(dim=-1)          # 마지막 시점의 예측

print("각 시점의 다음-아이템 점수 shape:", logits.shape)
print("마지막 시점이 예측한 다음 아이템 ID:", next_item.item(),
      "(학습 전이라 무작위에 가깝습니다)")
```

코드 해설

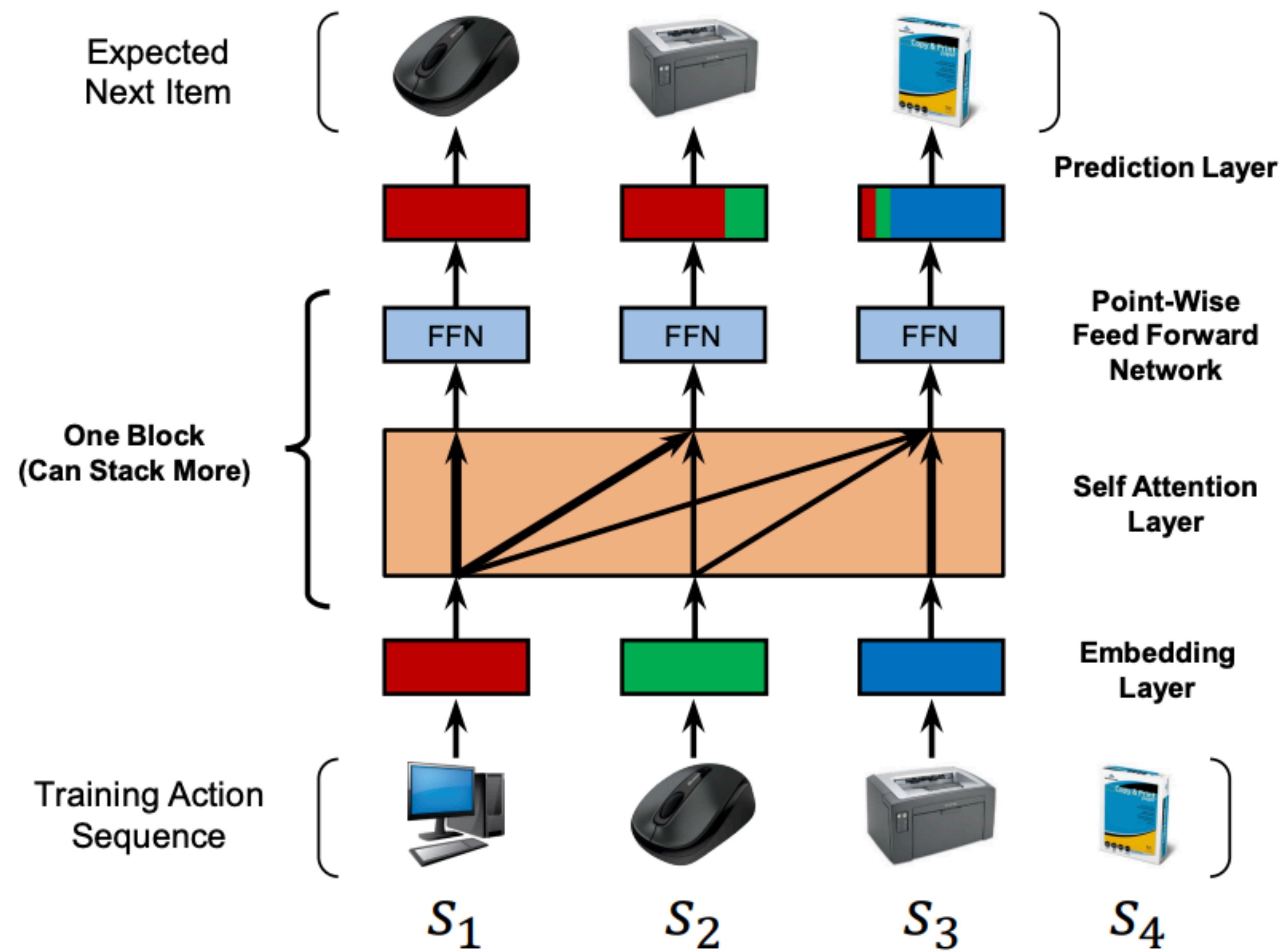
- **nn.Linear(d_model, num_items)** : 각 시점의 출력 벡터를 '전체 아이템 수만큼의 점수'로 변환합니다. 가장 점수가 높은 아이템이 모델의 다음-아이템 예측입니다.
- **logits[:, -1]** : 시퀀스의 **마지막 시점** 출력만 뽑아, '이 사용자가 다음에 볼 아이템'을 예측합니다. (학습 중에는 모든 시점의 출력을 동시에 사용해 '각 시점의 다음 아이템'을 맞히도록 훈련합니다 — lab_03에서 실제로 학습합니다.)
- 지금은 **학습 전**이라 예측이 무작위에 가깝습니다. lab_03에서 실제 MovieLens 데이터로 이 구조를 학습시켜 의미 있는 추천을 만들어 봅니다.

💡 **직접 해보기**: **num_heads** 를 4로, **num_layers** 를 1로 바꾸면 출력 shape과 가중치가 어떻게 달라지는지 확인해보세요. (단, **d_model** 은 **num_heads** 로 나누어떨어져야 합니다.)

3. SASRec

- 구현 방식에 따른 모델의 결과 확인하기

Self-Attentive Sequential Recommendation



Q & A

End of Document